

Campo	Valor
Repositório GitHub	github.com/bru-compressores/compras
URL em produção	https://compras-iscv.onrender.com
Pasta local	C:\Users\gusta\Downloads\controle-compras_1\compras
Banco de dados	Supabase — projeto "Controle Compras BRU"
Hospedagem	Render (free tier) — serviço "compras" / projeto "BRU Compressores"
Contas (todas corporativas)	compras@brucompressores.com.br / bru-compressores@brucompressores.com.br

1. IDENTIFICAÇÃO E OBJETIVO

Sistema web interno para **gestão completa do processo de compras de peças para Ordens de Serviço (O.S.)** da BRU Compressores. Resolve o problema de falta de rastreabilidade, visibilidade e controle sobre o ciclo de compra — desde a chegada da O.S. ao departamento até a entrega final.

O criador do sistema (Gustavo) está saindo da empresa. Há um sucessor de perfil intermediário assumindo. Todos os documentos de transição já foram gerados.

2. CONTEXTO OPERACIONAL

- A BRU realiza manutenção em compressores — quando uma O.S. é aprovada pelo cliente, é exportada do ERP Primam como PDF
- O almoxarifado faz triagem: separa peças em estoque e envia apenas os itens faltantes para compras
- Compras cota, fecha preço com fornecedores e acompanha a entrega
- O sistema importa PDFs de O.S. (dois layouts do Primam suportados) e de Pedidos de Compra
- Todos os PDFs vêm do ERP Primam — nunca de outras fontes

3. ESTADO ATUAL DO PROJETO

CONCLUÍDO E FUNCIONANDO

- ✓ Login com JWT, dois papéis (admin/operador)
- ✓ Dashboard com KPIs, alertas e semáforos de prazo
- ✓ Listagem de O.S. com filtros, busca ILIKE (case-insensitive) e paginação
- ✓ Detalhe da O.S. com tabela de peças (18 colunas sincronizadas)
- ✓ Importação de PDF de O.S. (Layouts 1 e 2 do Primam)
- ✓ Importação de PDF de Pedido de Compra (vincula peças automaticamente)
- ✓ Fluxo de Triagem completo com badge de notificação no menu
- ✓ Kanban com 6 colunas e drag-and-drop

- ✓ Avanço automático de status da O.S. (peças → O.S.)
- ✓ Status automático da O.S. baseado em peças especiais (Bloqueado/Ag. Aprovação)
- ✓ Rastreamento de peças
- ✓ Saving de Compras com abas (Saving | Lead Times)
- ✓ Lead Times automáticos capturados por evento
- ✓ CRUD de Fornecedores com autocomplete
- ✓ Relatórios de economia e histórico de preço
- ✓ Backup/Restauração JSON + importação de fornecedores via Excel
- ✓ Configurações: markup sinaleiro + KPIs do dashboard (drag-and-drop)
- ✓ Gestão de usuários (admin/operador)
- ✓ PDF gerado pelo sistema para impressão da O.S.
- ✓ Toggle "Ocultar separados/entregues" na tabela de peças
- ✓ Dropdown visual de status com badges coloridos no modal de edição
- ✓ Responsividade mobile completa (sidebar drawer, modais bottom sheet)
- ✓ Exclusão em lote de O.S. na listagem

ÚLTIMA TAREFA REALIZADA

Busca na listagem de O.S. não encontrava clientes nem equipamentos — só números. Causa: **LIKE é case-sensitive no PostgreSQL**. Corrigido para ILIKE em backend/routes/ordens.js. Confirmado funcionando.

PENDÊNCIAS / PRÓXIMOS PASSOS

Nenhuma pendência definida. Sistema estável. Aguardar novas solicitações.

4. REGRAS DE NEGÓCIO CRÍTICAS

Status das Peças — Fluxo Completo

Status	Quando ocorre
Aguardando Triagem	Status inicial ao importar PDF da O.S.
Separado (Almoxarifado)	Decisão na triagem: almox tem em estoque — NÃO entra no Rastreamento
Pendente	Decisão na triagem: precisa comprar
Em cotação	Compras buscando preços
Bloqueado no Fornecedor	Pendência financeira com fornecedor
Aguardando Aprovação Técnica	Aguardando confirmação técnica do depto
Pedido realizado	PC emitido — captura data_primeiro_pedido se for o 1º
Em trânsito	Mercadoria a caminho
Entregue	Recebido
Cancelado	Cancelado

REGRA: Avanço Automático da O.S. ← DEFINIDA PELO USUÁRIO

- **Aberta** → **Aguardando peças**: quando TODAS as peças ativas (exceto Separado, Cancelado, Aguardando Triagem) estiverem com status: Pedido realizado, Em cotação, Em trânsito ou Entregue
- **Aguardando peças** → **Peças separadas**: quando TODAS as peças ativas estiverem Entregues

■ `ativas.length === 0` NÃO deve disparar avanço — **vacuous truth bug** do JS (`[]every()` retorna true). Proteção já implementada.

REGRA: Status Automático da O.S. por Peças ← DEFINIDA PELO USUÁRIO

- Se qualquer peça = "Aguardando Aprovação Técnica" → O.S. muda para esse status (**prioridade máxima**)
- Se qualquer peça = "Bloqueado no Fornecedor" (sem Ag. Aprovação) → O.S. muda para esse status
- Quando todas resolvidas → O.S. volta para "**Aberta**" automaticamente
- Toda mudança é registrada no `historico_status`

REGRA: Saving ← DEFINIDA PELO USUÁRIO

Saving = **Preço Cotado** – **Valor Fechado** (NÃO usa Preço de Venda do ERP). Exibido em R\$ e em %.

REGRA: Markup Sinaleiro

Fórmula: **Preço de Venda (ERP) ÷ Valor Fechado**

■ Verde: $\geq \text{markup_verde}$ (padrão 2.2x) | ■ Laranja: entre `markup_laranja` e `markup_verde` | ■ Vermelho: $< \text{markup_laranja}$ (padrão 2.0x)

REGRA: Lead Times — Captura Automática por Evento

Campo	Quando é capturado
-------	--------------------

data_entrada_compras	Ao importar PDF da O.S.
data_triagem_concluida	Ao finalizar triagem
data_primeiro_pedido	Quando 1ª peça vai para Pedido realizado
data_todas_pedidas	Quando O.S. avança para Aguardando peças
data_entrega_completa	Quando O.S. avança para Peças separadas

REGRA: Colunas da Tabela de Peças — 18 COLUNAS SINCRONIZADAS

■ **_colsPecas e células DEVEM ter exatamente o mesmo número. 18 colunas = 18 células + 1 botão excluir = 19 total. Sempre verificar após mudanças.**

#	Coluna	Dado
1	Código	codigo
2	Referência	referencia
3	Cód. Fabricante	codigo_fabricante
4	Descrição	descricao
5	Qtd	quantidade
6	Venda	preco_unitario
7	Cotado	preco_cotado
8	Fechado	preco_fechado
9	Markup	calculado
10	Data RC	data_entrada_compras da O.S.
11	Data Compra	data_compra da peça (emissão PC)
12	Prev. Entrega	data_entrega_prevista (semáforo)
13	Fornecedor	fornecedor_nome
14	Nº PC	numero_pc (badge azul)
15	Status	dropdown visual
16	Transporte	transporte
17	Rastreio	numero_rastreio
18	Observações	observacoes

5. ESTRUTURA DO BANCO DE DADOS

Tabela: ordens_servico

Coluna	Tipo / Descrição
id	SERIAL PK
numero_os	TEXT UNIQUE
cliente	TEXT
equipamento	TEXT
data_abertura	TEXT
data_conclusao_estimada	TEXT
status	TEXT
prioridade	TEXT (Alta/Média/Baixa)
tipo	TEXT (OS/Pedido)
data_entrada_compras	TIMESTAMP — capturado ao importar PDF
data_triagem_concluida	TIMESTAMP
data_primeiro_pedido	TIMESTAMP
data_todas_pedidas	TIMESTAMP
data_entrega_completa	TIMESTAMP

Tabela: pecas_os

Coluna	Tipo / Descrição
id	SERIAL PK
os_id	INTEGER FK → ordens_servico
codigo	TEXT
referencia	TEXT (ex: KX500-007)
codigo_fabricante	TEXT (ex: KA600-030TB)
descricao	TEXT NOT NULL
quantidade	NUMERIC(10,3) — aceita decimais como 0.5 KG
preco_unitario	NUMERIC (preço de venda ERP)
preco_cotado	NUMERIC
preco_fechado	NUMERIC
fornecedor_id	INTEGER FK → fornecedores
status_entrega	TEXT
numero_pc	TEXT (ex: PC-1820)
data_compra	TEXT (emissão do PC)

data_entrega_prevista	TEXT
numero_rastreio	TEXT
transporte	TEXT
observacoes	TEXT

Tabela: fornecedores

Coluna	Tipo / Descrição
id	SERIAL PK
nome	TEXT NOT NULL
cnpj/telefone/email/cidade/estado	TEXT

Tabela: historico_status

Coluna	Tipo / Descrição
id	SERIAL PK
os_id	INTEGER FK → ordens_servico
status_anterior/status_novo	TEXT
observacao	TEXT
usuario_id	INTEGER FK

Tabela: usuarios

Coluna	Tipo / Descrição
id	SERIAL PK
nome/email	TEXT
senha_hash	TEXT (bcrypt)
papel	TEXT (admin/operador)
ativo	INTEGER 1/0

Tabela: configuracoes

Coluna	Tipo / Descrição
chave	TEXT PK: markup_verde, markup_laranja, dashboard_ordem, dashboard_ocultos
valor	TEXT

6. ARQUITETURA TÉCNICA

Componente	Tecnologia	Detalhe
Frontend	Vanilla JavaScript SPA	Uma página por arquivo em /frontend/assets/js/pages/. Sem framework.
Backend	Node.js 18+ / Express	API REST. Todas as rotas em /backend/routes/. Auth JWT.
Banco produção	PostgreSQL via Supabase	Shared Pooler PORTA 6543. NUNCA usar porta 5432 (IPv6 only).
Banco local	sql.js (SQLite em memória)	Apenas para desenvolvimento local. Não usado em produção.
Hospedagem	Render (free tier)	Deploy automático via GitHub. Dorme após 15min — normal.
Repositório	GitHub	github.com/bru-compressores/compras — branch main

Variáveis de Ambiente (configurar no Render → Environment)

```
DATABASE_URL =  
postgres://postgres.XXXXX:SENHA@aws-1-sa-east-1.pooler.supabase.com:6543/postgres  
JWT_SECRET = BRUcompressores@2026#SecretKey!xyz789
```

Padrão CRÍTICO de Queries — NUNCA omitir Promise.resolve()

```
const q = (db, sql, ...p) => Promise.resolve(db.prepare(sql).get(...p)); // 1 linha  
const qa = (db, sql, ...p) => Promise.resolve(db.prepare(sql).all(...p)); // array  
const qr = (db, sql, ...p) => Promise.resolve(db.prepare(sql).run(...p)); // insert/update/delete  
  
// Rotas SEMPRE async/await:  
router.get('/', async (req, res) => {  
  try {  
    const db = getDB();  
    const os = await q(db, 'SELECT * FROM ordens_servico WHERE id = ?', id);  
    res.json(os);  
  } catch(e) { res.status(500).json({ erro: e.message }); }  
});
```

PostgreSQL retorna números como STRING — sempre converter

```
const total = parseInt(result.total) || 0;  
const valor = parseFloat(result.valor) || 0;  
const mk = f.markup_medio ? parseFloat(f.markup_medio).toFixed(2) : '-';
```

Processo de Deploy

```
xcopy /E /Y "C:\Users\gusta\Downloads\controle-compras_X\compras\*" "  
"C:\Users\gusta\Downloads\controle-compras_1\compras\  
cd C:\Users\gusta\Downloads\controle-compras_1\compras  
git add .  
git commit -m "descricao da mudanca"  
git push --force
```

■ git push --force é o padrão deste projeto. Push normal rejeita.

7. BUGS CONHECIDOS E PONTOS DE ATENÇÃO

database.js revertendo para versão antiga [CRÍTICO]

Sintoma: Sintoma: Erro "Use getAsync no PostgreSQL" nos logs do Render.

Solução: Verificar este arquivo PRIMEIRO em qualquer diagnóstico. Ele tem histórico de regredir para versão síncrona.

Vacuous truth — [].every() retorna true

Sintoma: Sintoma: O.S. avança de status sem ter peças para comprar.

Solução: Sempre verificar `ativas.length > 0` ANTES de `.every()`. Proteção já implementada.

PostgreSQL retorna números como string

Sintoma: Sintoma: `.toFixed` is not a function ou NaN em cálculos.

Solução: Sempre `parseInt()` ou `parseFloat()` ao usar valores do banco.

Colunas vs células desalinhadas na tabela de peças

Sintoma: Sintoma: Dados aparecem nas colunas erradas.

Solução: 18 colunas definidas = 18 células + 1 botão excluir = 19 total. Verificar após qualquer mudança.

Mistura de pastas no deploy

Sintoma: Causa: Extrair `compras-main.zip` gera pasta `compras-main/` — copiar pasta inteira.

Solução: Sempre usar `xcopy /E /Y "pasta\*" (note o \* no final — copia conteúdo, não pasta)`

Constantes regex fora da função extrairOSPrimam

Sintoma: Sintoma: 0 peças importadas mesmo PDF sendo válido.

Solução: Todas as constantes regex DEVEM ficar DENTRO da função (problema com `eval` em modo de módulo).

Outros Pontos de Atenção

- Supabase pausa após 7 dias sem uso → reativar em `supabase.com` → "Resume project"
- Render dorme após 15 min → primeiro acesso demora ~30s (normal)
- Após formatar o PC: `git config --global user.email` e `user.name` precisam ser reconfigurados
- Busca usa `ILIKE` (não regredir para `LIKE` — case-sensitive no PostgreSQL)
- Quantidade é `NUMERIC(10,3)` — não regredir para `INTEGER` (quebra PDFs com 0.5 KG)
- Tela de triagem usa `triagem.js` — arquivo que já foi perdido uma vez (não estava em ZIP antigo)

8. O QUE NÃO PODE SER ALTERADO

Item	Motivo
Porta 6543 para Supabase	Render free não suporta IPv6 (porta 5432) — sistema não conecta sem isso
Promise.resolve() em todas as queries	Compatibilidade SQLite (local) + PostgreSQL (produção)
Saving = Cotado – Fechado	Definido pelo usuário — cálculos e relatórios dependem disso
Busca com ILIKE	LIKE é case-sensitive no PostgreSQL
Quantidade NUMERIC(10,3)	Aceita decimais como 0.5 KG — já alterado no banco
18 colunas sincronizadas na tabela	Desalinhamento causa bugs visuais sérios
Prioridade: Ag. Aprovação > Bloqueado	Definido pelo usuário
O.S. volta para Aberta ao resolver	Definido pelo usuário — comportamento automático
Triagem: Compras que marca	Almoxarifado informa por fora — fluxo desenhado assim
git push --force	Padrão do projeto — push normal rejeita
Constantes regex dentro de extrairOSPrimam	Problema de escopo com eval em módulos Node.js

9. DESIGN / UI / UX

Badges de Status das Peças

Status	Cor fundo	Cor texto
Aguardando Triagem	#fef3c7 (amarelo)	#92400e
Separado (Almoxarifado)	#dcfce7 (verde)	#166534
Pendente	#f1f5f9 (cinza)	#64748b
Em cotação	#ede9fe (lilás)	#7c3aed
Bloqueado no Fornecedor	#fff7ed (laranja)	#c2410c — .badge-bloqueado
Aguardando Aprovação Técnica	#f5f3ff (roxo)	#7c3aed — .badge-aprovacao
Pedido realizado	#eff6ff (azul)	#1a56db
Em trânsito	#fffbeb (âmbar)	#d97706
Entregue	#f0fdf4 (verde escuro)	#16a34a
Cancelado	#fef2f2 (vermelho)	#dc2626

Highlights de Linha na Tabela de Peças

- Separado (Almoxarifado) → fundo verde claro + borda esquerda verde
- Aguardando Triagem → fundo amarelo claro + borda esquerda âmbar

Responsividade Mobile

Sidebar: drawer deslizante com overlay. Modais: bottom sheet. Tabelas: scroll horizontal. KPIs: 2 colunas. Menu toggle exibido automaticamente em telas $\leq 768\text{px}$.

Dropdown de Status nas Linhas da Tabela

position: fixed com coordenadas calculadas do botão (evita overflow). Detecta espaço: abre para cima se sem espaço abaixo. Fecha ao clicar fora.

10. COMO USAR ESTE PACOTE NA NOVA CONVERSA

1. Baixe o código atual: github.com/bru-compressores/compras/archive/refs/heads/main.zip
2. Abra uma nova conversa com o Claude em claude.ai
3. Cole o conteúdo completo deste pacote (versão .md) no início da conversa
4. Anexe o ZIP baixado do GitHub
5. Descreva o que precisa — o Claude assumirá o projeto sem explicações adicionais

■ O código real está no GitHub — este pacote é o contexto. Ambos são necessários.